

Security Report

TableMaster Pro — v1.3.56

Datum: 26 March 2026 | Bestand: tablemaster-pro.zip | Reviewer: Claude (Anthropic)

Dit rapport beschrijft de resultaten van een statische code-analyse van de WordPress-plugin **TableMaster Pro v1.3.56**. Alle PHP-bronbestanden, het frontend-template en de updater zijn handmatig beoordeeld op veelvoorkomende kwetsbaarheden.

Overzicht gevonden issues

#	Issue	Ernst	Bestand
1	Server-Side Request Forgery (SSRF)	HOOG	class-settings.php class-updater.php
2	Externe updater zonder handtekeningverificatie	HOOG	class-updater.php
3	Stored XSS via inline_html instelling	GEMIDDELD	templates/table-frontend.php
4	CSS-injectie risico in <style> tag	GEMIDDELD	templates/table-frontend.php
5	Blocking sleep() in AJAX-handler	GEMIDDELD	class-updater.php

#1 Server-Side Request Forgery (SSRF)

● HOOG

Betrokken bestanden: class-settings.php, class-updater.php

De instelling `update_url` is door een beheerder aanpasbaar via het instellingenpaneel. De validatie controleert enkel op HTTPS en een geldige hostnaam, maar blokkeert geen interne of private IP-adressen. De server maakt vervolgens een HTTP-verzoek naar die URL:

```
$response = wp_remote_get( $url, array(
    'timeout' => 30,
    'sslverify' => true,
    'headers' => array( 'X-License-Key' => $this->license_key ),
) );
```

Een aanvalleur met beheerderstoegang kan hiermee de webserver requests laten sturen naar interne diensten zoals `https://localhost/admin` of `https://192.168.1.1/`.

Aanbeveling:

- Gebruik `wp_http_validate_url()` in combinatie met expliciete blokkering van loopback- en private IP-ranges.
- Overweeg de `update_url` hardcoded te maken en niet via de UI aanpasbaar.

#2 Externe updater zonder handtekeningverificatie

● HOOG

Betrokken bestand: class-updater.php

De plugin haalt update-informatie op van een externe server (standaard: `table-importer-tool.replit.app`) en vertrouwt de JSON-respons volledig zonder enige cryptografische verificatie:

```
$decoded = json_decode( $body );  
// ...  
$obj->package = $this->get_download_url(); // download zonder signing
```

Als de externe server gecompromitteerd is — of als de `update_url` wordt gewijzigd (zie Issue 1) — kan een kwaadaardige plugin als legitieme update worden aangeboden en automatisch geïnstalleerd. De update-server draait bovendien op Replit, wat geen geschikte infrastructuur is voor beveiligingskritische software-updates.

Aanbeveling:

- Voeg een cryptografische handtekening (bijv. Ed25519) toe aan update-pakketten en verifieer deze vóór installatie.
- Verplaats de update-server naar een betrouwbare, gecontroleerde infrastructuur.
- Gebruik een vastgelegde publieke sleutel in de plugin om handtekeningen te verifiëren.

#3 Stored XSS via `inline_html` instelling

● GEMIDDELD

Betrokken bestand: templates/table-frontend.php

Wanneer de instelling `inline_html` is ingeschakeld voor een kolom, wordt celinhoud via `wp_kses_post()` weergegeven. Deze functie laat HTML toe die een 'post editor' mag gebruiken, waaronder potentieel misbruikbare tags:

```
} elseif ( $col_type === 'html' && $inline_html ) :  
    echo wp_kses_post( $raw_content );  
}
```

Hoewel momenteel alleen beheerders (`manage_options`) tabeldata kunnen opslaan, is dit een latent risico. Als in een toekomstige versie redacteurrollen toegang krijgen, is Stored XSS mogelijk.

Aanbeveling:

- Beperk de toegestane HTML-tags voor `inline_html` verder met een aangepaste `$allowed_tags` array in `wp_kses()`.
- Documenteer expliciet dat deze functie alleen voor beheerders bedoeld is.

#4 CSS-injectie risico in <style> tag

● GEMIDDELD

Betrokken bestand: templates/table-frontend.php

CSS-regels worden dynamisch in een <style>-tag geschreven. De lettergrootte-waarden worden momenteel correct gewhitelisted, maar de waarden worden daarna uitgevoerd via `esc_attr()`:

```
$rules[] = 'font-size:' . esc_attr( $f['size'] );  
// ...  
echo implode( ';', $rules );
```

`esc_attr()` is **niet** voldoende voor contextbescherming binnen een CSS-blok — het ontsnapt enkel HTML-attributen. Als de whitelist ooit uitgebreid wordt met vrije invoer, is CSS-injectie (en daarmee potentieel XSS via CSS expressions) mogelijk.

Aanbeveling:

- Gebruik `esc_attr()` niet voor CSS-waarden; gebruik in plaats daarvan strikte whitelist-validatie voor alle CSS-eigenschappen.
- Overweeg CSS-waarden te genereren via een veilige, gecontroleerde mapping in plaats van directe uitvoer.

#5 Blocking sleep() in AJAX-handler

● GEMIDDELD

Betrokken bestand: class-updater.php

De update-check bevat een retry-loop met een blokkerende `sleep()` van 3 seconden per poging, met een maximum van 3 pogingen:

```
for ( $attempt = 0; $attempt <= $this->max_retries; $attempt++ ) {  
    if ( $attempt > 0 ) {  
        sleep( $this->retry_delay ); // 3 seconden  
    }  
    // ...  
}
```

Dit kan resulteren in **tot 9 seconden blokkerende PHP-executie** per request. Meerdere gelijktijdige admin-sessies of herhaalde aanroepen door een aanvaller kunnen PHP-workers vastzetten en zo een Denial-of-Service veroorzaken.

Aanbeveling:

- Verplaats de update-check naar een WordPress Cron-job (`wp_schedule_event()`) in plaats van een synchrone AJAX-aanroep.
- Verwijder de `sleep()` aanroepen uit synchrone request-handlers.

Positieve bevindingen

De plugin toont over het algemeen een serieuze beveiligingsaanpak. De volgende maatregelen zijn correct geïmplementeerd:

Maatregel	Beschrijving	Locatie
Nonce + capability check	Centrale verify_nonce() op elke AJAX-actie	class-ajax.php
SQL-injectiepreventie	\$wpdb->prepare() consequent gebruikt	class-db.php
Input sanitization	intval(), sanitize_text_field(), wp_kses_post()	Overal
Output escaping	esc_attr(), esc_html(), esc_url() in templates	table-frontend.php
ABSPATH check	Bovenaan elk PHP-bestand aanwezig	Alle bestanden
Rate limiting	Max 30 AJAX-verzoeken/minuut per gebruiker	class-ajax.php
Bestandsgrootte-limieten	1MB kolommen, 10MB rijen bij opslaan	class-ajax.php
Whitelist validatie	Enumeraties voor types, uitlijning, sortering	class-ajax.php / class-db.php

Dit rapport is gegenereerd op basis van statische code-analyse. Dynamische of penetratietests zijn niet uitgevoerd. De ernst-scores zijn gebaseerd op de OWASP-risicoclassificatie.